

Infrastructure and server specifications

1CibiPlatform

Audience: engineering management and infrastructure approvers.

Purpose: what we need to run the platform, and why each item is required.

1. Summary of requirements

#	Requirement	Specification	Why it is needed
1	Application server	8 vCPU / 16 GB RAM	Runs the entire platform in a single process
2	Database server	4 vCPU / 16 GB RAM, NVMe SSD	Holds orders, documents, AI search data and logs
3	Redis	2–4 GB RAM	Caching; required before adding a second server
4	Docker Hub	Paid team plan	Protects our application images and prevents failed deployments
5	Container resource limits	Configuration	Stops one component consuming the whole server
6	WebSocket support at the public entry point	Configuration	Required for real-time features to work

Sized for approximately 100–300 concurrent users. Figures are engineering estimates derived from the application's actual workload and should be reviewed against production data after go-live.

2. What we are running

1CibiPlatform is a **modular monolith**: one application process containing nine business modules (ATS, Auth, PhilSys, CNX, SSO, Employment Verification, AI Agent, Platform Logging, CBBlue). This is a deliberate design choice — simpler and cheaper to operate than microservices — with one consequence for sizing:

All nine modules share the same memory and CPU. We size one capable server rather than several small ones.

Component	Role
API	The application: all nine business modules
Gateway	Single public entry point, routing and rate limiting

PostgreSQL	All business data, plus application logs
Redis	Caching

3. Application server — 8 vCPU / 16 GB

Why this size:

- **Nine modules in one process.** Memory requirements add together rather than being isolated.
 - **Continuous background work.** Three scheduled jobs run permanently: bulk candidate submission and email notification every 10 seconds, applicant search indexing every minute. The server is never fully idle.
 - **Document generation is memory-intensive.** Producing PDF reports and Excel files creates large, short-lived memory spikes. This is the most common cause of out-of-memory failures in systems like ours, and the main reason for 16 GB rather than 8 GB.
 - **AI features are inexpensive here.** The AI assistant calls an external provider over the network, so it uses very little CPU or memory on our server and does not materially affect this sizing.
-

4. Database server — 4 vCPU / 16 GB, NVMe SSD

The database is the component most likely to become the bottleneck, and where additional memory delivers the clearest return.

Why this size:

- **AI search data.** The platform stores mathematical representations of documents used for AI search. These are large and must remain in memory to perform well.
- **Constant background polling.** The scheduled jobs above query the database every 10 seconds, around the clock.
- **Reporting queries.** Order and report searches scan significant volumes of history.
- **Application logs are stored here** (see §6).

NVMe SSD is not optional. Database performance is dominated by disk speed; standard disks will make the platform feel slow regardless of CPU or memory.

Recommendation: run the database on a separate server or a managed database service. Sharing a single machine means a burst of document generation can starve the database and slow the entire platform.

5. Docker Hub — paid team plan

Our application images are published to Docker Hub automatically by the build pipeline on every release to development, UAT and production.

Risk on the free tier	Business impact
-----------------------	-----------------

Free accounts are rate limited on image downloads	Deployments fail at the worst time — during an urgent fix, when we deploy repeatedly
Free repositories are public	Our application images would be downloadable by anyone
No availability guarantee	A failed download blocks both releases and rollbacks

Private repositories are the more important half of this. Our images contain compiled application code and must not be publicly accessible.

A paid team plan provides private repositories, removes download limits, and adds seat management and access control. This is a low-cost, high-consequence item: inexpensive, but without it a release can be blocked at a critical moment.

6. Logging — no additional database required

Application logs are written to PostgreSQL by a purpose-built module:

- batched writing, so logging never slows down user requests;
- stored in a separate **logging** schema, kept apart from business data;
- automatic cleanup of logs older than **10 days**;
- a built-in query interface for reviewing logs.

A team member reviews logs regularly, so issues are identified while they are recent. A 10-day window is sufficient for troubleshooting and keeps the log table small enough that it never competes with order and report data for database memory.

This is a **troubleshooting** window, not an audit trail. Business history — orders, status changes and report activity — is stored permanently in the ATS tables and is unaffected by this setting.

Why no separate logging database: logging is already batched and asynchronous, so a different database product would not produce a difference users could perceive. It would add a second system to back up, patch, secure and monitor, plus roughly 2 vCPU / 8 GB of additional server cost — while PostgreSQL would still be required for orders and AI search.

If log volume ever becomes a problem, the cheapest first step is moving the log table to its own PostgreSQL instance, reusing everything already built.

7. Important constraint: scale up, not out

The platform currently supports **one application server**. Running two or more would cause real-time notifications — bulk upload completion messages and AI assistant responses — to be delivered intermittently, with **no error shown to the user or recorded in the logs**. Features would simply appear to fail some of the time.

This is a known limitation with a small, well-defined fix (approximately a one-line configuration change, using the Redis server we already run). It has not been done because we do not yet need multiple servers.

Practical guidance:

- To handle more load now, **increase the size of the existing server**.

- Before adding a **second** application server, this fix must be completed and tested. It is a half-day change, not a project — but it must not be skipped.

8. Recommended environments

Environment	Application	Database
Development / UAT	4 vCPU / 16 GB (combined)	shared with application
Production	8 vCPU / 16 GB	4 vCPU / 16 GB, NVMe

Additional production requirements:

- **Container resource limits.** None are currently configured, which means one component can consume all server resources and slow everything else. This should be set before go-live.
- **Database tuning.** PostgreSQL is running default settings intended for small machines. It must be tuned for the allocated memory to realise the benefit of the larger server.
- **WebSocket support at the public entry point.** Real-time features require the public web server to permit WebSocket connections. This is not currently configured correctly in the development environment and must be verified in production.

9. Summary of asks

1. **Production application server:** 8 vCPU / 16 GB.
2. **Production database:** 4 vCPU / 16 GB with NVMe storage, ideally separate or managed.
3. **Docker Hub paid team plan** — protects our application images and prevents deployment failures.
4. **Container resource limits and database tuning** configured before go-live.
5. **Do not add a second application server** until the real-time notification fix is completed.

Prepared from analysis of the application code and deployment configuration. Figures are engineering estimates and should be validated against production load once monitoring data is available.